WEEKLY MINI PROJECT 03 · TEAM 4

Who Doesn't Know?

Work Package · Backend A

Backend | NestJS + MongoDB

PROJECT	TEAM LEADER	OWNER
Who Doesn't Know?	**Taima Watfeh**	**Ali Al Hamwi**

ROLE	AREA
Backend developer	**Game lifecycle · Secret tips · Bootstrap**

Everything before the first question: the room, the players, the roles and what everyone may see.

Use the same class pattern: keep this work inside `src/games/` and `src/tips/`. Each feature has its own `*.module.ts`, `*.controller.ts`, `*.service.ts`, `dto/` and `schemas/`. The controller exposes routes, DTOs validate input, and the service talks to Mongoose and enforces rules.

1 Your Work at a Glance

#	Task	Main file(s)
T1	Bootstrap: validation, Swagger, CORS	`main.ts`
T2	Secret tips seed	`tips/tips.seed.ts`
T3	InternalToolGuard + tips endpoints	`tips/*`
T4	Create game	`games/*`
T5	Join game	`games/*`
T6	Start game	`games/*`
T7	Private role view `/me`	`games/*`
T8	Public state `/state` + read-service stub	`games/state.builder.ts`
T9	Logger events	`games/games.service.ts`

Use the class pattern: copy the structure you already built in `projects` and `workspaces`. Each feature gets its own module, controller, service, DTOs and schema. Keep the controller thin; put rules in the service; return clear NestJS exceptions.

2 Your Tasks, Step by Step

T1 Bootstrap

`backend/src/main.ts`

1. `app.useGlobalPipes(new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true }))`.
2. Swagger at `/api` with tags `games`, `gameplay`, `tips`.
3. `app.enableCors()` for the frontend URL.

✔ Done when: Swagger opens and a request with an unknown field returns 400.

T2 Secret tips seed

`backend/src/tips/tips.seed.ts` · `backend/src/tips/tips.service.ts`

1. Write 32+ tips in 4+ categories: Places, Food, Jobs, Sports (8 each).
2. In `onModuleInit`: if `secret_tips` is empty, insert them with numeric ids; use fixed seed ids or the same last-id pattern inside this service.

✔ Done when: a fresh database contains 30+ active tips in 4+ categories.

T3 InternalToolGuard + tips endpoints

`backend/src/tips/internal-tool.guard.ts` · `backend/src/tips/tips.controller.ts` · `backend/src/tips/dto/*`

1. Guard (approach taught in class): compare the internal-tool header to `INTERNAL_TOOL_TOKEN`; otherwise throw.
2. Endpoints with the guard: `GET /tips`, `POST /tips`, `PATCH /tips/:id`, `PATCH /tips/:id/deactivate`.
3. DTOs: `CreateTipDto { text, category }`, `UpdateTipDto` (optional fields).
4. Swagger: `@ApiHeader` for the header on each tips endpoint.

✔ Done when: in Swagger, creating a tip without the header fails and with it succeeds.

T4 Create game — POST /games

`backend/src/games/games.controller.ts` · `games.service.ts` · `dto/create-game.dto.ts` · `utils/codes.ts`

1. `CreateGameDto { displayName }`: required, trimmed, not empty.
2. `generateRoomCode()`: 5 chars from `A–Z0–9`; retry while a non-finished game uses it.
3. `generatePlayerId()`: `'p_' + randomBytes(6).toString('hex')`.
4. Create the game (`status: 'lobby'`) and the host player; return `{ gameId, roomCode, playerId, playerRef }`.

✔ Done when: Swagger returns a room code and a private player id.

T5 Join game — POST /games/join

`games.service.ts` · `dto/join-game.dto.ts`

1. `JoinGameDto { roomCode, displayName }`.
2. Room code not found → `404 "Game not found"`. Status not `lobby` → `409`.
3. `normalizedName = displayName.trim().toLowerCase()`; already used → `409`. Max players: 10 (constant, no rule depends on it).
4. Create the player and return the same response shape as create.

✔ Done when: duplicate names and late joins are rejected with clear messages.

T6 Start game — POST /games/:roomCode/start

`games.service.ts` · `dto/player-action.dto.ts`

1. Body `{ playerId }`; player must be in this game and be the host.
2. Not `lobby` → 409. Fewer than 4 players → 400 with "N more players needed".
3. Pick a random hidden player and a random active tip; shuffle the turn order (Fisher–Yates).
4. Save: `status 'questioning'`, `roundNumber 1`, `currentTurnIndex 0`, `turnsCompletedInRound 0`, `decisionNumber 1`, `votingNumber 1`.

✔ Done when: start with 3 is rejected, with 4 succeeds, second start returns 409.

T7 Private role view — GET /games/:roomCode/me?playerId=

`games.controller.ts` · `games.service.ts`

1. Unknown `playerId` or from another game → reject.
2. Return `PrivateRoleView`: `role 'waiting'` in lobby; `'informed'` + `secretTip`; `'hidden'` + `secretTip: null`.
3. `hasDecided` / `hasVoted` from the current `decisionNumber` / `votingNumber`.

✔ Done when: informed players get the same tip; the hidden player gets the "you don't know" state.

T8 Public state — GET /games/:roomCode/state

`games/state.builder.ts` · `gameplay/gameplay-read.stub.ts`

1. Build `PublicGameState` field by field (never spread a document). Never include `tempId`, `hiddenPlayerRef`, `secretTipId` before finished.
2. Compute `playersNeeded`, `currentPlayerRef`, `nextPlayerRef`, turns completed / remaining.
3. Log, pending question, decision / voting progress, vote counts come from `GameplayReadService`. Write the stub now (empty values); Mikdad replaces it.
4. When finished, fill `result`.

```
// backend/src/gameplay/gameplay-read.service.ts  (frozen interface)
export interface GameplayReadService {
  getLog(gameId: number): Promise;
  getPendingQuestion(gameId: number): Promise<{ id; askerRef; targetRef; text } | null>;
  getDecisionProgress(game: GameDoc): Promise;
  getVotingProgress(game: GameDoc): Promise;
  getVoteCounts(gameId: number, votingNumber: number): Promise<{ ref: number; votes: number }[]>;
}
// Ali Al Hamwi ships a stub that returns empty values; Ali Mikdad replaces the body. Same
signature.
```

✔ Done when: the state in Swagger shows no secret, no hidden player and no temporary ids.

T9 **Logger events**

`games.service.ts`

1. `Logger.log` on game created (room code) and game started (number of players).
2. Never log the tip or the hidden player. Don't log every request.

✔ Done when: two clean log lines per game lifecycle.

3 Rules for You

- Only edit `main.ts`, `games/*`, `tips/*` and the gameplay read stub. Follow the existing module/controller/service/dto/schema layout.
- You don't need Mikdad: the stub makes lobby and start fully testable. You don't need the frontend: test in Swagger.
- Every error is 400 / 404 / 409 with a human-readable message.

4 Presentation Part

Your part in the English presentation: **room codes, player ids, why the temporary id is not authentication, and the tips guard**. Prepare 1–2 minutes and be ready to explain your own code.